

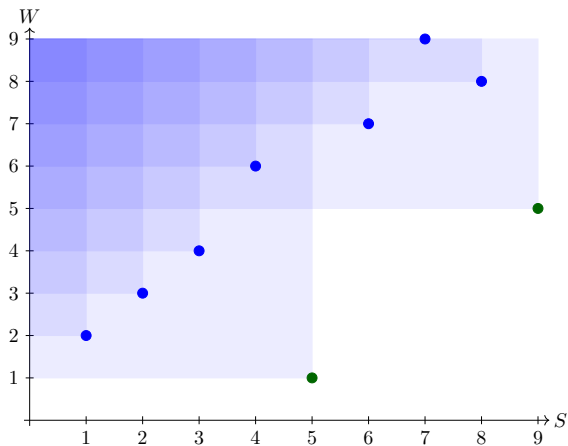
Solution for Aqueducts

Hugo Peyraud-Magnin

June 28, 2025

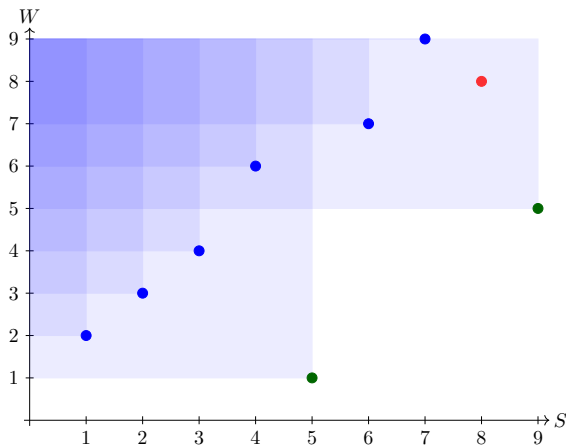
Statement

i dominates j iff $S_i > S_j$ and $W_i < W_j$.



Statement

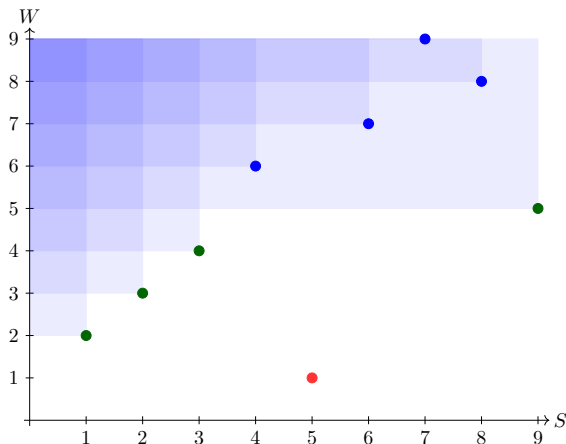
i dominates j iff $S_i > S_j$ and $W_i < W_j$.



For each material k , count the number of optimal (non-dominated) materials after removing k .

Statement

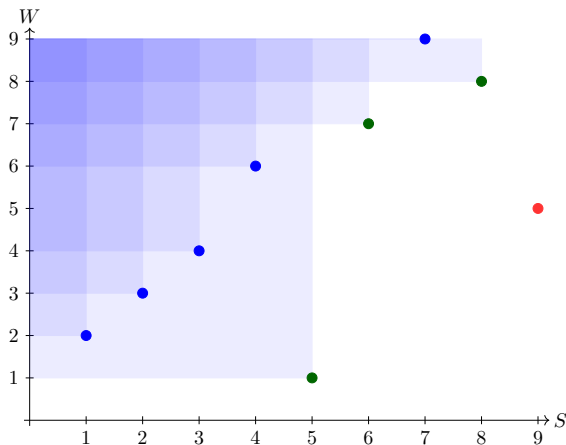
i dominates j iff $S_i > S_j$ and $W_i < W_j$.



For each material k , count the number of optimal (non-dominated) materials after removing k .

Statement

i dominates j iff $S_i > S_j$ and $W_i < W_j$.



For each material k , count the number of optimal (non-dominated) materials after removing k .

Subtask 1 ($N \leq 100$)

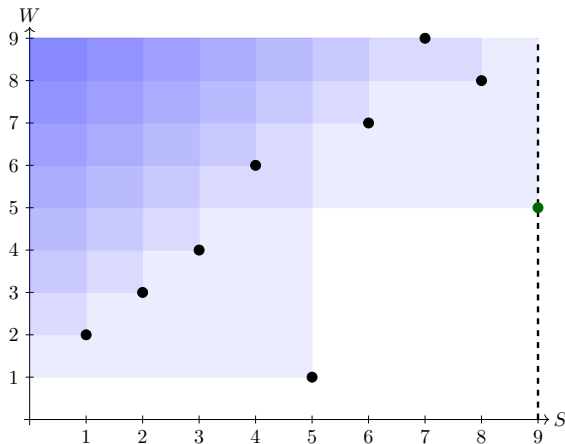
- ▶ For every k
 - ▶ For all $i, j \in \{0, 1, \dots, N-1\} \setminus \{k\}$
 - ▶ If i dominates j , mark j as bad.
 - ▶ C_k is the number of unmarked materials.

For every k , computing C_k takes $\mathcal{O}(N^2)$. Overall time complexity: $\mathcal{O}(N^3)$.

Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

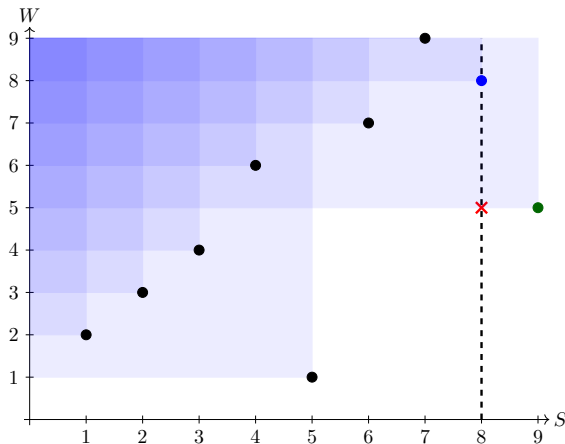
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

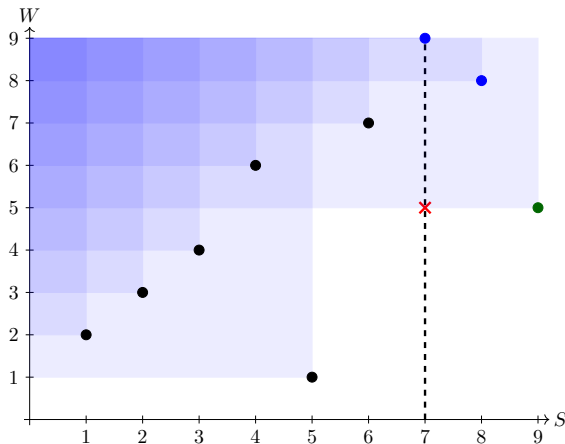
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

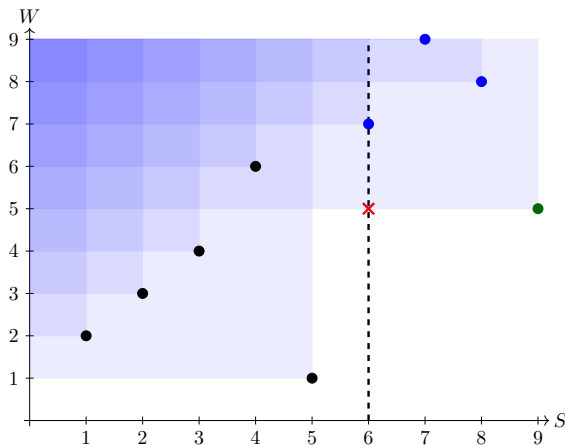
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

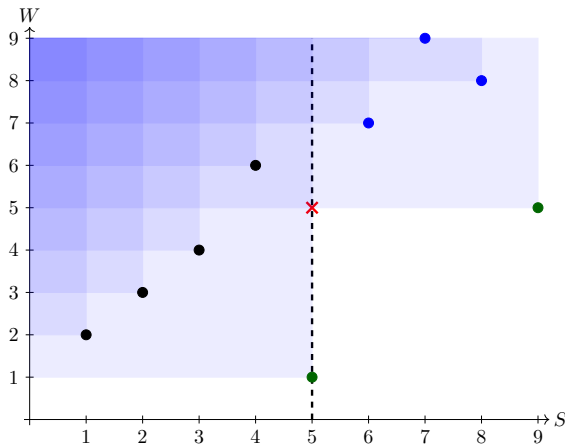
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

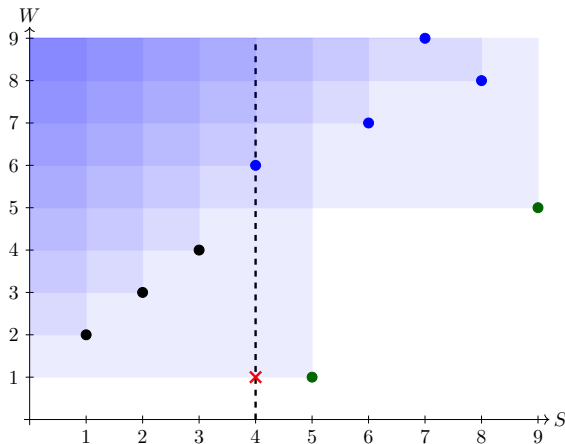
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

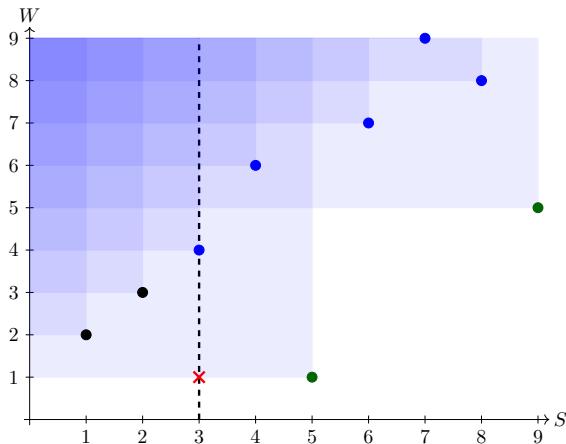
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

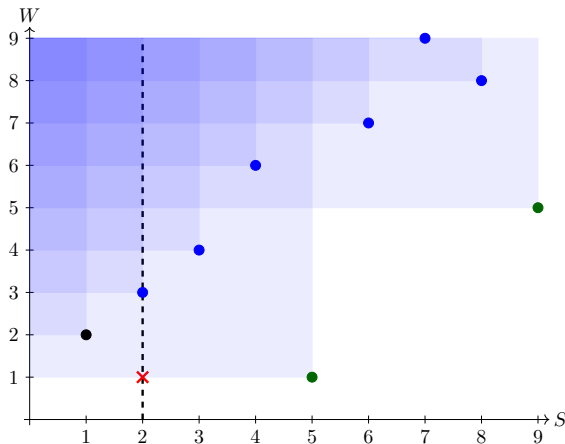
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

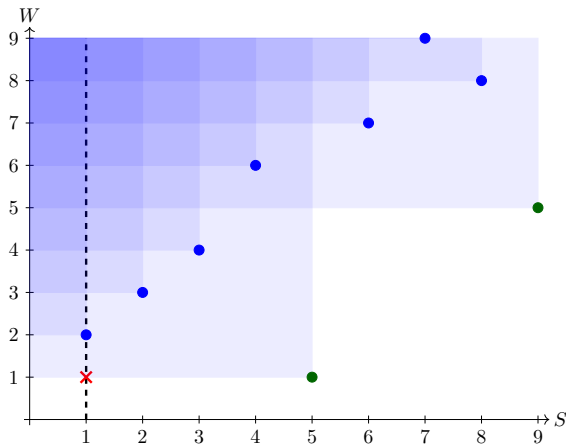
Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.



Subtask 2 ($N \leq 5000$)

Observation: a material is optimal iff it is lower than every other material to its right.

Algorithm: iterate over materials sorted by decreasing strength, while maintaining the minimum weight.

First, sort the materials in $\mathcal{O}(N \log N)$. Then for every k , compute C_k in $\mathcal{O}(N)$. Total complexity: $\mathcal{O}(N^2)$.

Subtask 3 ($|S_i - W_i| \leq 5$)

Observation: i can only be dominated by materials j with $S_j \in \{S_i + 1, \dots, S_i + 10\}$.

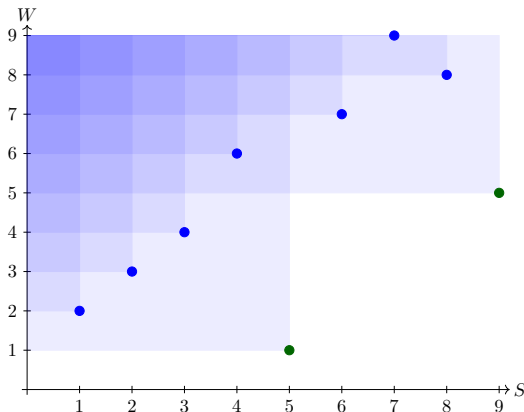
We can make an array or a map that associates each weight with the corresponding material (if it exists).

For every k , we can iterate over the (at most 10) materials dominated by k , and check for each of them if it's dominated by another material than k .

Subtask 4

Observation: Let T the number of optimal materials among the whole set of materials, and Δ_i the number of materials which are dominated only by i .

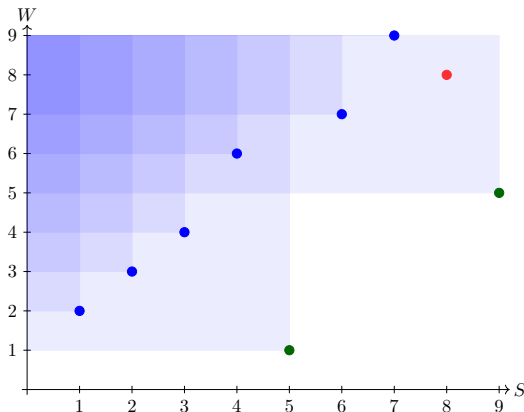
$$C_k = \begin{cases} T & \text{if } i \text{ was not optimal} \\ T - 1 + \Delta_k & \text{otherwise} \end{cases}$$



Subtask 4

Observation: Let T the number of optimal materials among the whole set of materials, and Δ_i the number of materials which are dominated only by i .

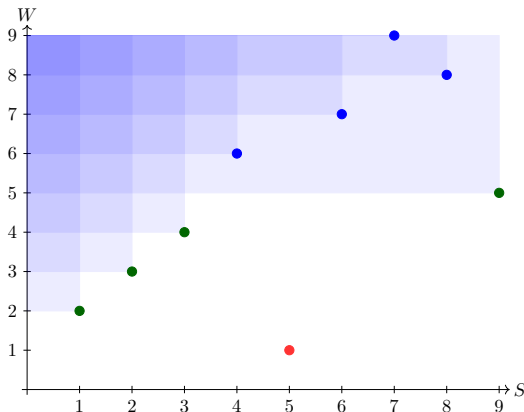
$$C_k = \begin{cases} T & \text{if } i \text{ was not optimal} \\ T - 1 + \Delta_k & \text{otherwise} \end{cases}$$



Subtask 4

Observation: Let T the number of optimal materials among the whole set of materials, and Δ_i the number of materials which are dominated only by i .

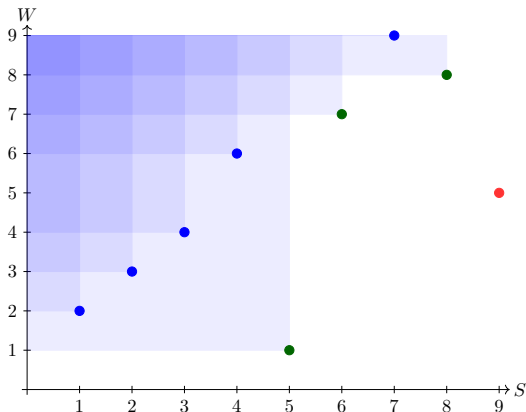
$$C_k = \begin{cases} T & \text{if } i \text{ was not optimal} \\ T - 1 + \Delta_k & \text{otherwise} \end{cases}$$



Subtask 4

Observation: Let T the number of optimal materials among the whole set of materials, and Δ_i the number of materials which are dominated only by i .

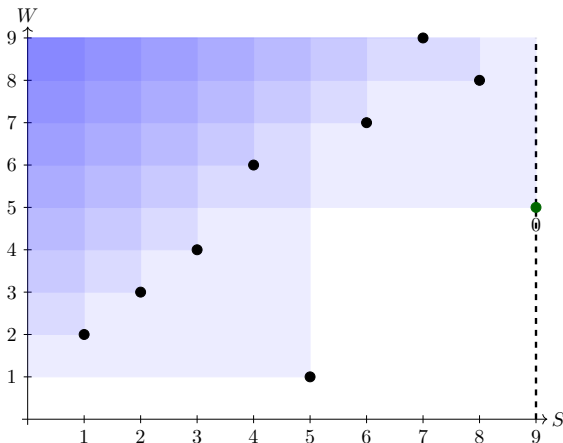
$$C_k = \begin{cases} T & \text{if } i \text{ was not optimal} \\ T - 1 + \Delta_k & \text{otherwise} \end{cases}$$



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

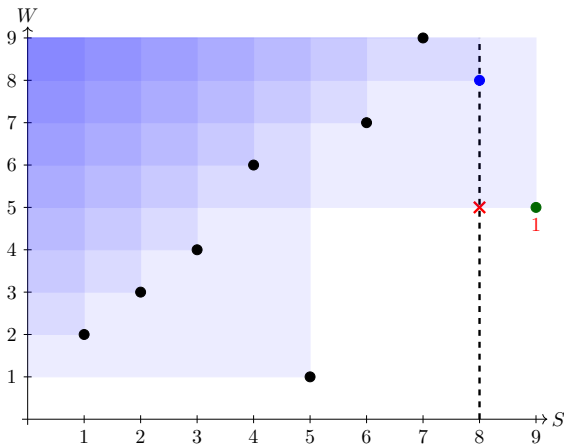
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

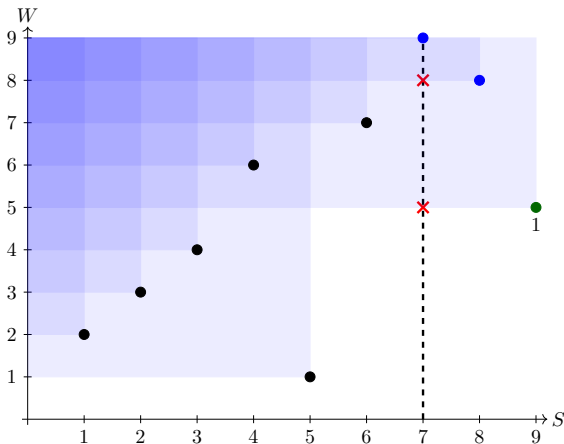
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

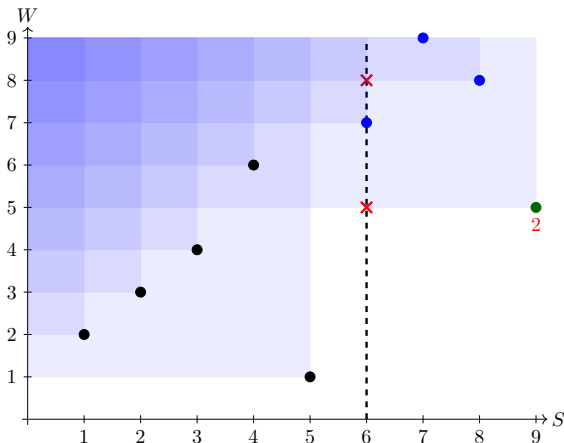
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

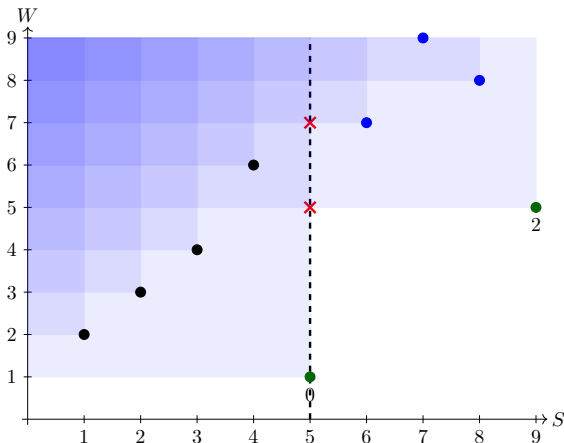
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

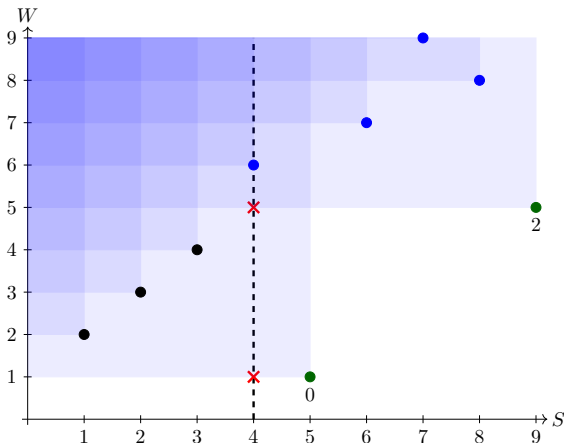
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

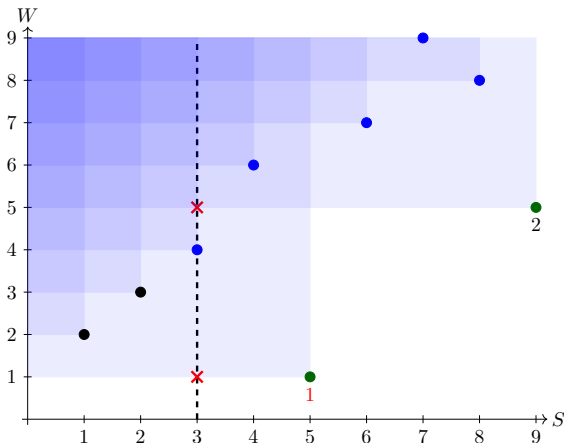
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

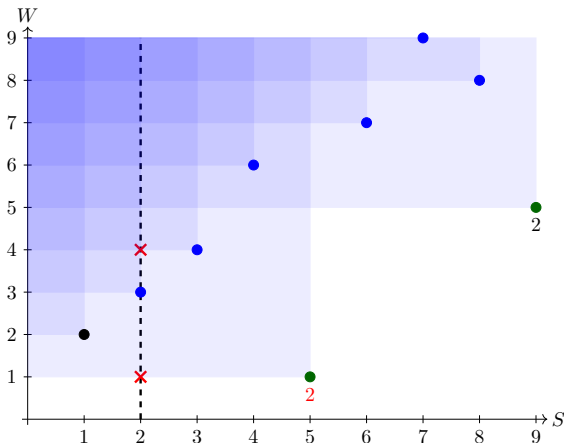
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

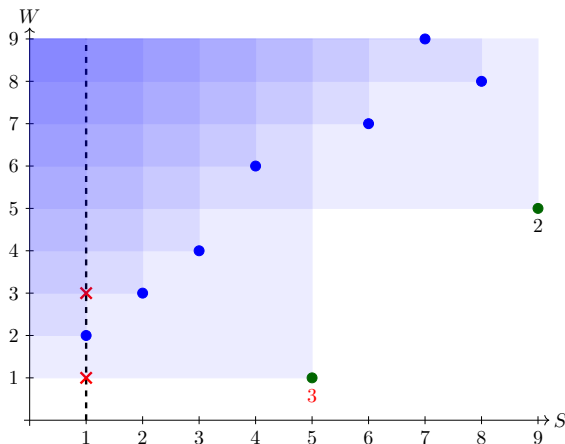
When the weight of the current material is between the two minimums, increment $\Delta i_{\min}$.



Subtask 4

Algorithm: iterate over materials by decreasing weight, while maintaining the two minimum weights, and $i_{\min}$ the index of the material with minimum weight.

When the weight of the current material is between the two minimums, increment $\Delta_{i_{\min}}$. After sorting, it runs in $\mathcal{O}(N)$.



Senate Building Decoration

Mikhail Pyaderkin

Two-player game. Given an array, the players repeatedly remove **everything but a subarray at least half the length**, until only one element is left.

The first player wants to **maximize** the final number, while the second player wants to **minimize** it.

Subtask: A is nondecreasing

Clearly, out of all subarrays of a given length L , the first player will want to choose the rightmost one and the second player the leftmost one.

Also, it is never useful to give out a subarray that is longer than needed: the opponent is always able to give back a new array that contains smaller numbers than if we gave them the shortest array.

Thus, we can repeatedly cut off the left or right halves to find the last remaining position. This can be easily implemented in $n \log n$ time.

Subtask: $n \leq 250$ (n^4)

We can do **dynamic programming** with $O(n^2)$ states: for each subarray, remember what's the highest value you would get if player 1 started from that subarray, and the lowest value if player 2 started from that subarray.

In $O(n^2)$, for every state we can check the corresponding value for the other player for all its subarrays, for a total cost of $O(n^4)$

Subtask: $n \leq 800$ (n^3)

We can still do the same dynamic programming solution, but we can compute the minimum/maximum value for each length of subarray in $O(n)$ with a Range Minimum Query structure.

This gives a total cost of $O(n^3)$.

Full solution ($n^2 \log n$)

Let's **binary search** on the final value in the array. We now need to figure out whether the final value is $\geq$ or $<$ of the target.

This is equivalent to solving the problem on a A array that is either 0 or 1.

To solve this special case, we just need to keep track of the **number** of 0s (or 1s) answers you can get to from a given configuration.

If we process sub-arrays in order of length, we can update these numbers when going from length l to length $l + 1$ for **all** sub-arrays of that length in $O(n)$.

This makes the total cost $O(n^2 \log n)$

Political Patricians

Félix Moreno Peñarrubia

There is an unknown partition of $[0, \dots, 999]$.

We can ask, for any partition, which elements do not have another element that appears in the same set in both partitions.

Only two sets

Suppose we query a partition made of pairs of elements.

If the elements of a pair are alone, they are in separate sets. If not, they are in the same set. Either way, we will know their relationship.

We can then ask a similar query using one element per pair, and repeat. After $\log_2 n$ queries, this gives us full knowledge of the unknown partition.

Suppose we split the full set into two parts, and ask a query in which each of the two parts are together. If an element ends up being alone, then its only friend is in the other half, otherwise it is in the same half.

We can ask $\log_2 n$ queries, splitting odd vs even, 0, 1 and 2, 3 mod 4, etc. This will let us figure out exactly where the friend of each element is.

Ask all pairs. For each pair, we can easily tell whether they are together or apart in the hidden set.

If we ask $\frac{n}{2}$ pairs at once, we ask all pairs in $\sim n$ queries.

$0.5 \log^2 n$ queries (~ 80 points)

Keep a bunch of sets $S_1, \dots, S_k$, each containing elements that we know to be internally distinct.

Initially we have $S_i = \{i\}$. We can use the solution of the “sets of pairs” subtask to merge each pair of consecutive S_i s, figuring out which elements belong to the same set in the hidden partition.

We then create new S_i sets keeping a single representative from each merged pair.

$0.25 \log^2 n$ queries (~ 98 points)

Since we know that S_i and S_{i+1} are not in the same set, we can do better:

1. First find which elements of $S_i \cup S_{i+1}$ appear twice
2. We now have two sets A and B that we know to be separate but to hit the same sets in the hidden partition. Compare the first halves A_0, B_0 of A and B , and the second halves A_1, B_1 . We can make eight more sets $A'[0, 1, 2, 3]$ and $B'[0, 1, 2, 3]$ as follows:
 - $A'[0], B'[0]$: alone(A_0), alone(B_1)
 - $A'[1], B'[1]$: non-alone(A_0), non-alone(B_0)
 - $A'[2], B'[2]$: alone(A_1), alone(B_0)
 - $A'[3], B'[3]$: non-alone(A_1), non-alone(B_1)
3. Repeat step 2, splitting all set pairs in parallel

On average, the size of sets reduces by a factor of 4 at every step.

We try to merge as many queries as possible.

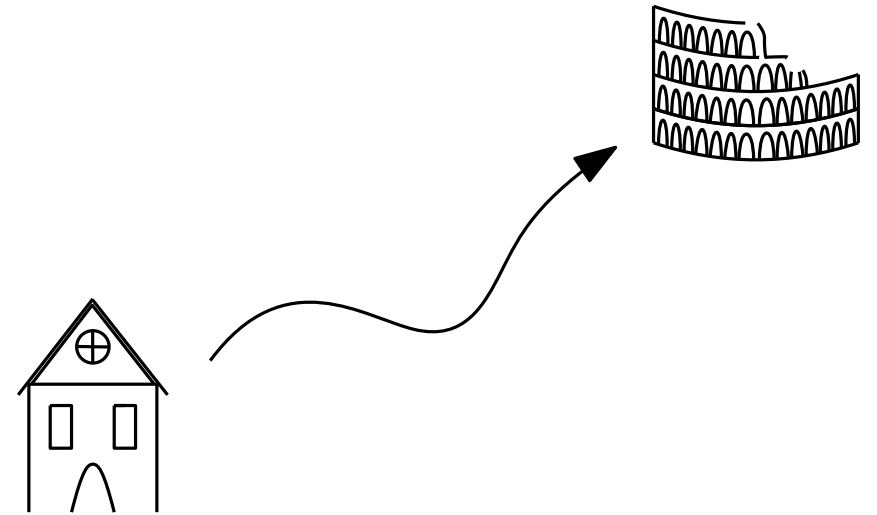
We first do all 1. queries from the $0.25 \log^2 n$ solution; then, we keep a stack of all pairs of sets from 2. that we still have to disambiguate and greedily add them to queries where their elements are not used yet.

This does fewer queries. In particular, for the “sets of pairs” subtask this is guaranteed to do at most $\log_2 n$ queries.

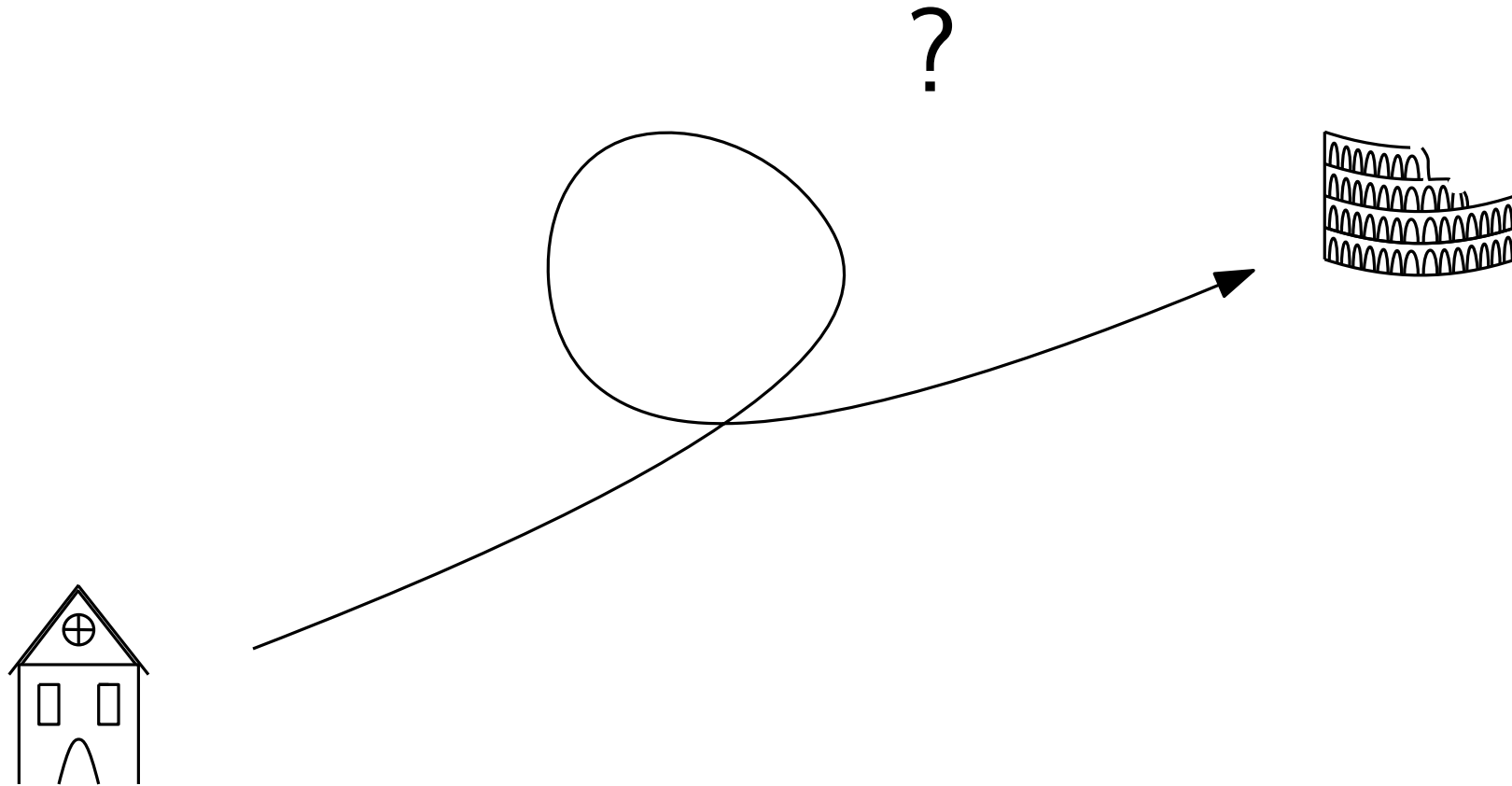
Road Reorganization

Luca Versari

WEOI 2025

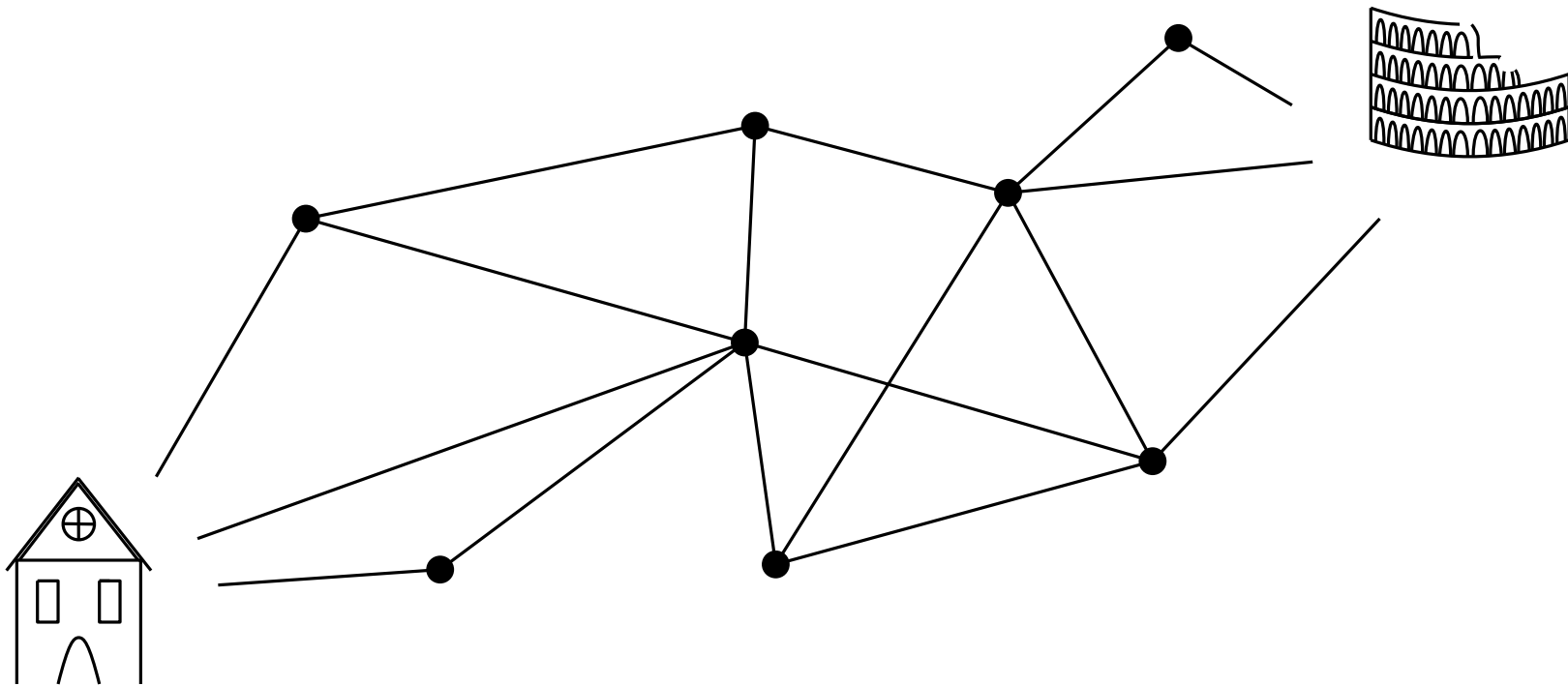


Finding the shortest path

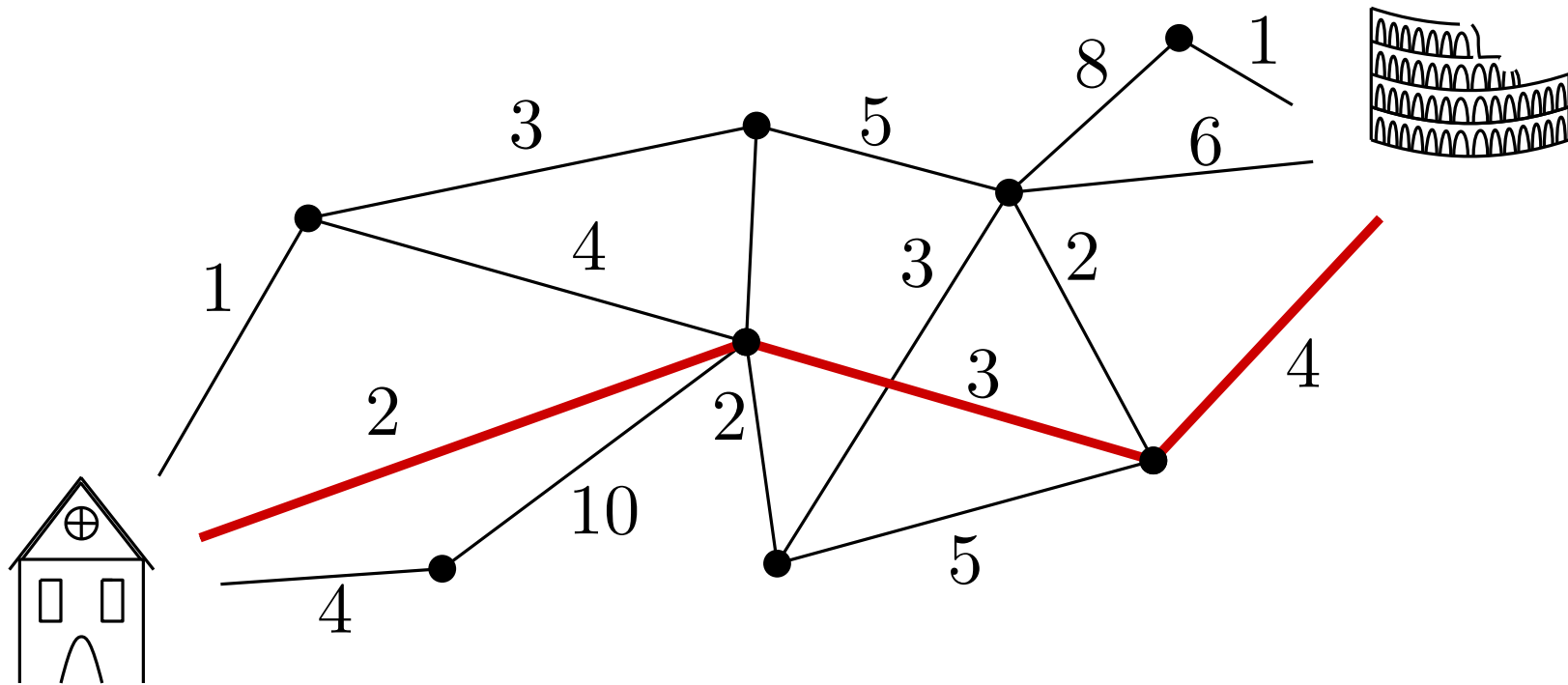


Use Dijkstra to find the shortest path in all subtasks

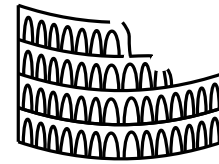
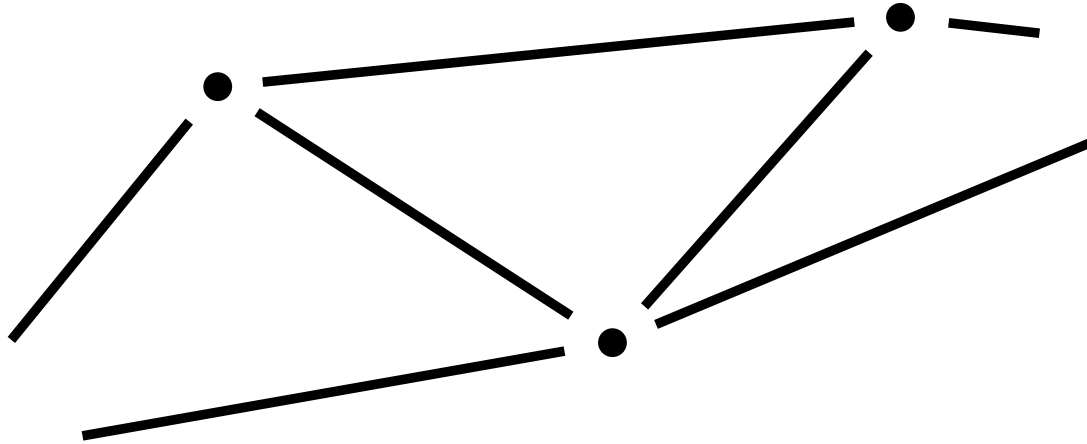
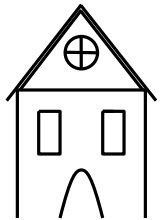
Finding the shortest path



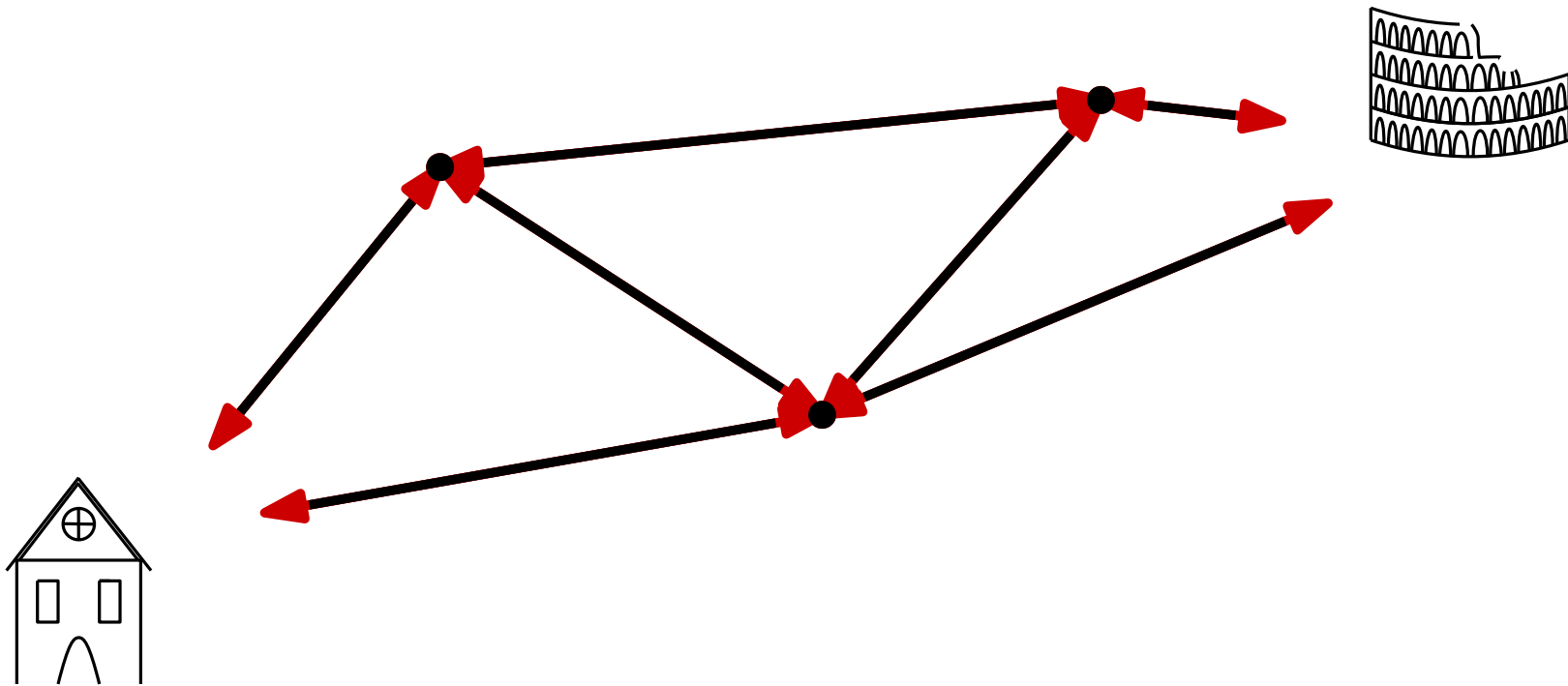
Finding the shortest path



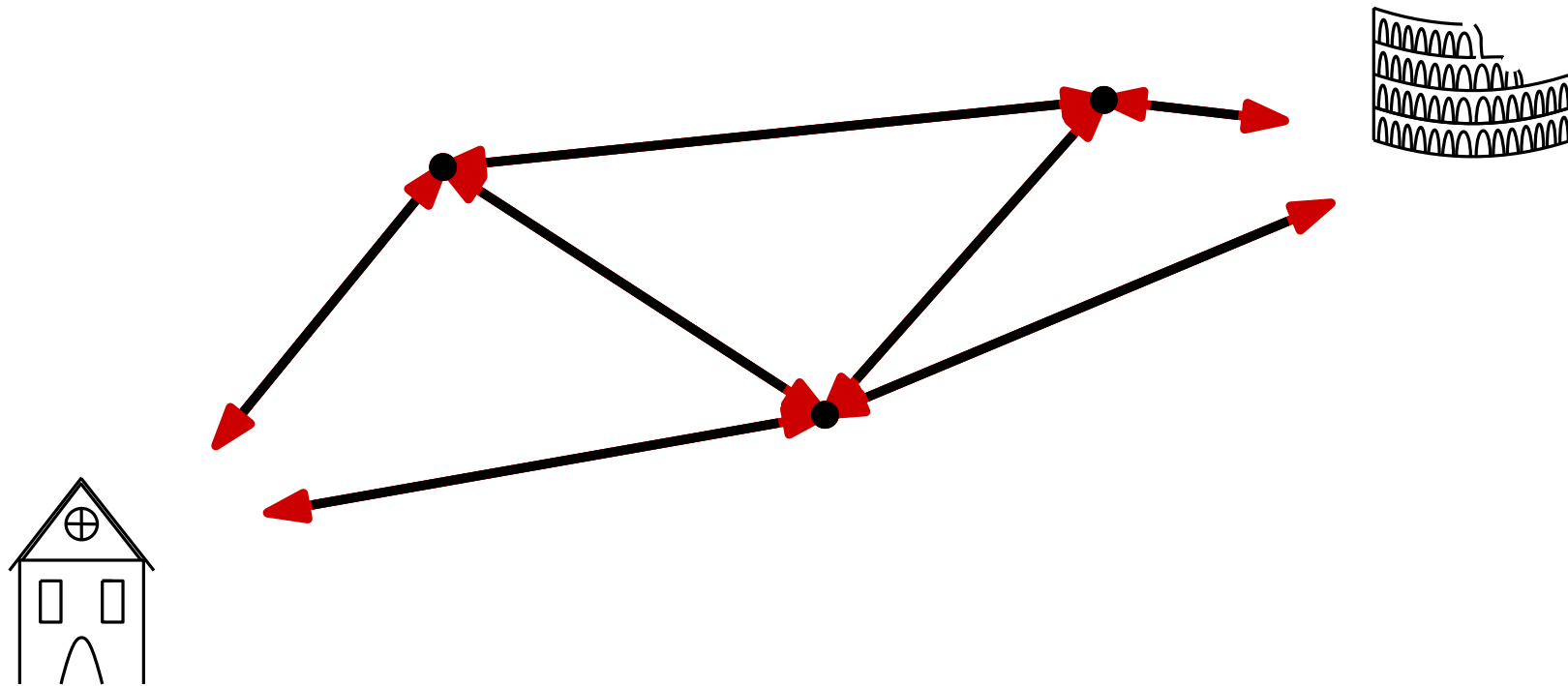
Subtask1: Brute Force



Subtask1: Brute Force



Subtask1: Brute Force



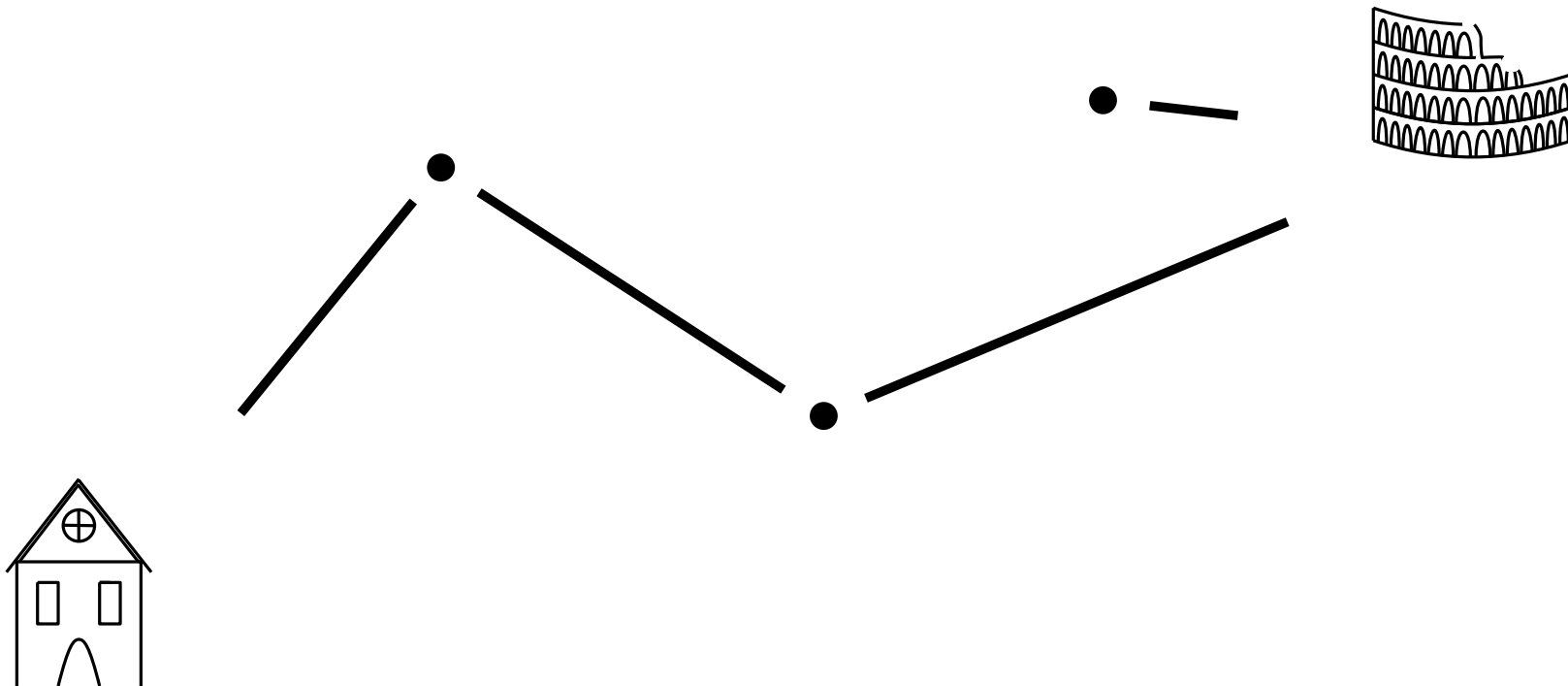
There are at most 20 edges

Try both direction for each edge gives 2^{20} options

Checking each of the options with DFS gives $\sim 20\,000\,000$ operations

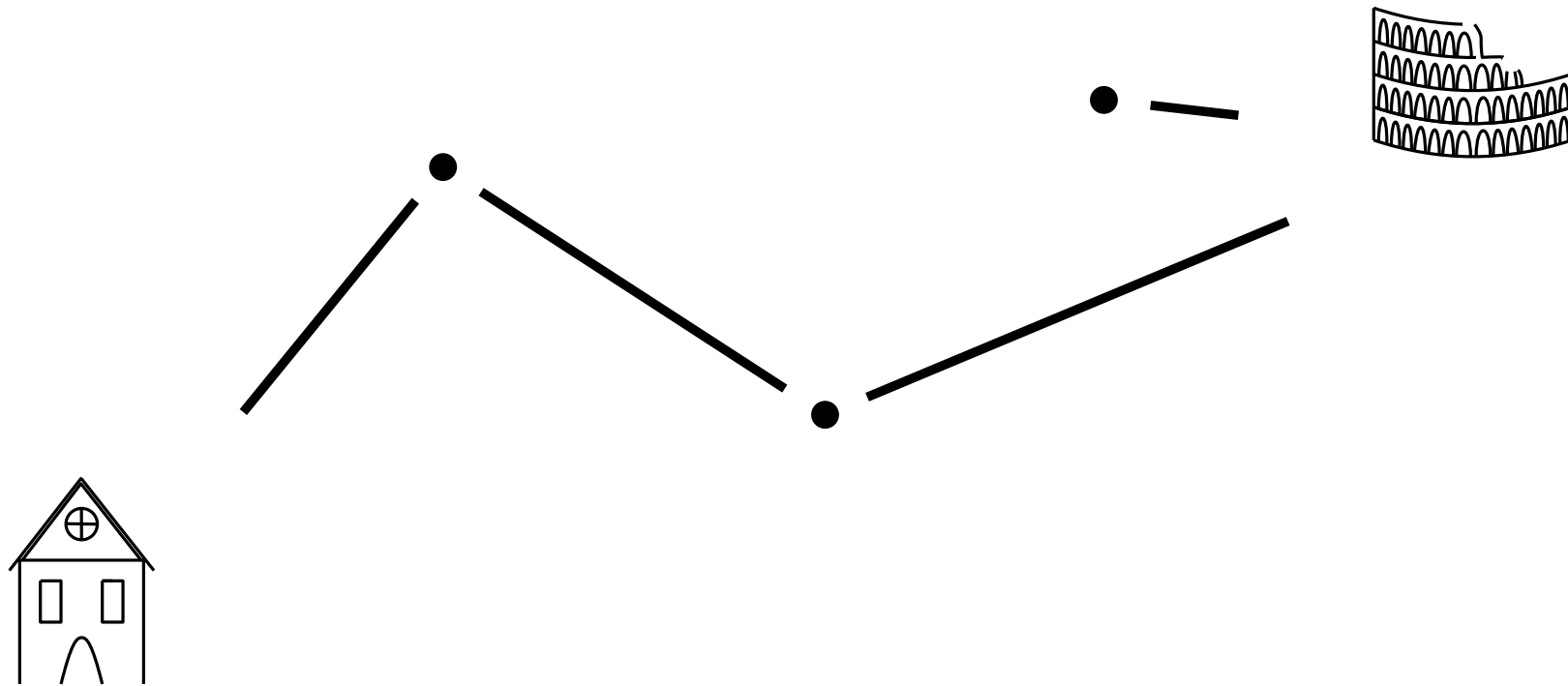
Subtask2: $N \leq M$

If the graph is a tree:



Subtask2: $N \leq M$

If the graph is a tree:



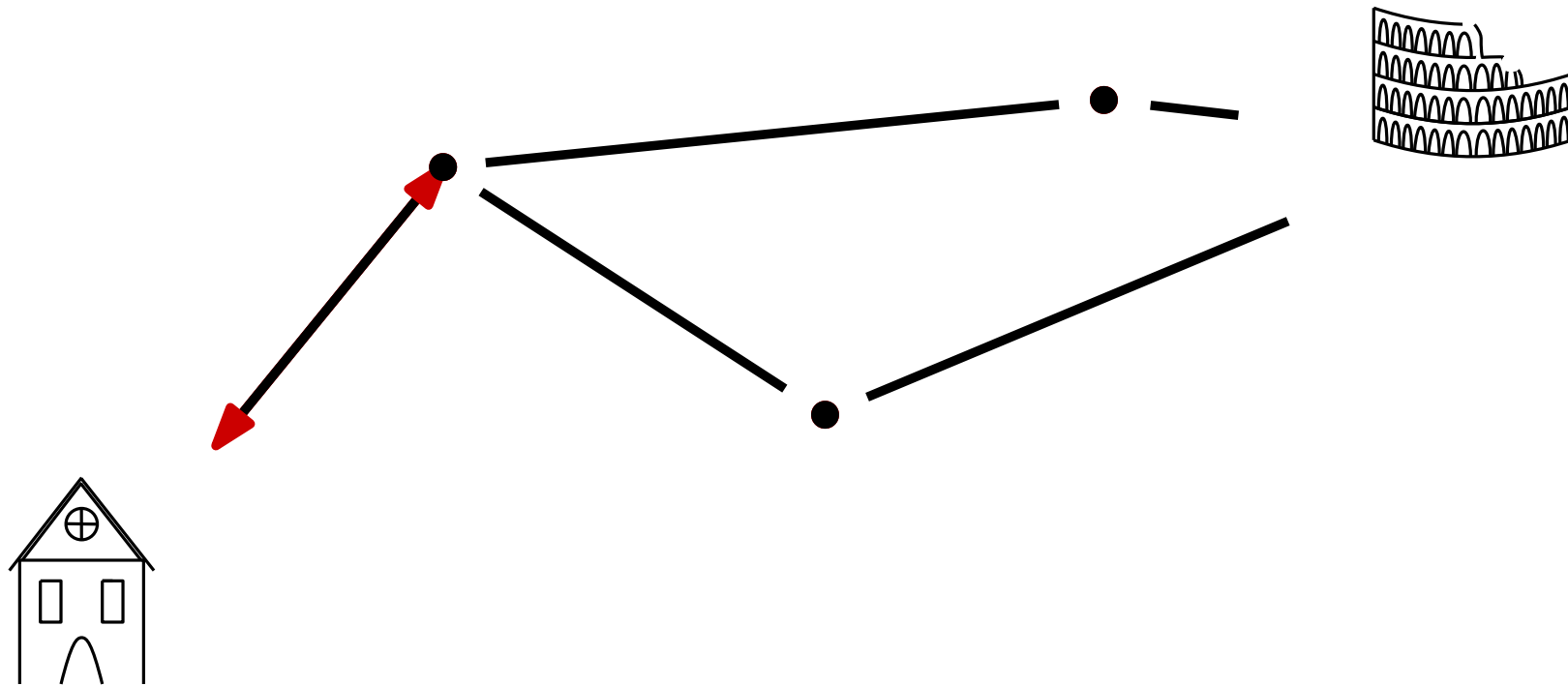
In a tree there is a unique path between each pair of vertices

Thus there is a unique path from Home to Rome, which is also the shortest

Directing this towards Rome makes it impossible to get back home

Subtask2: $N \leq M$

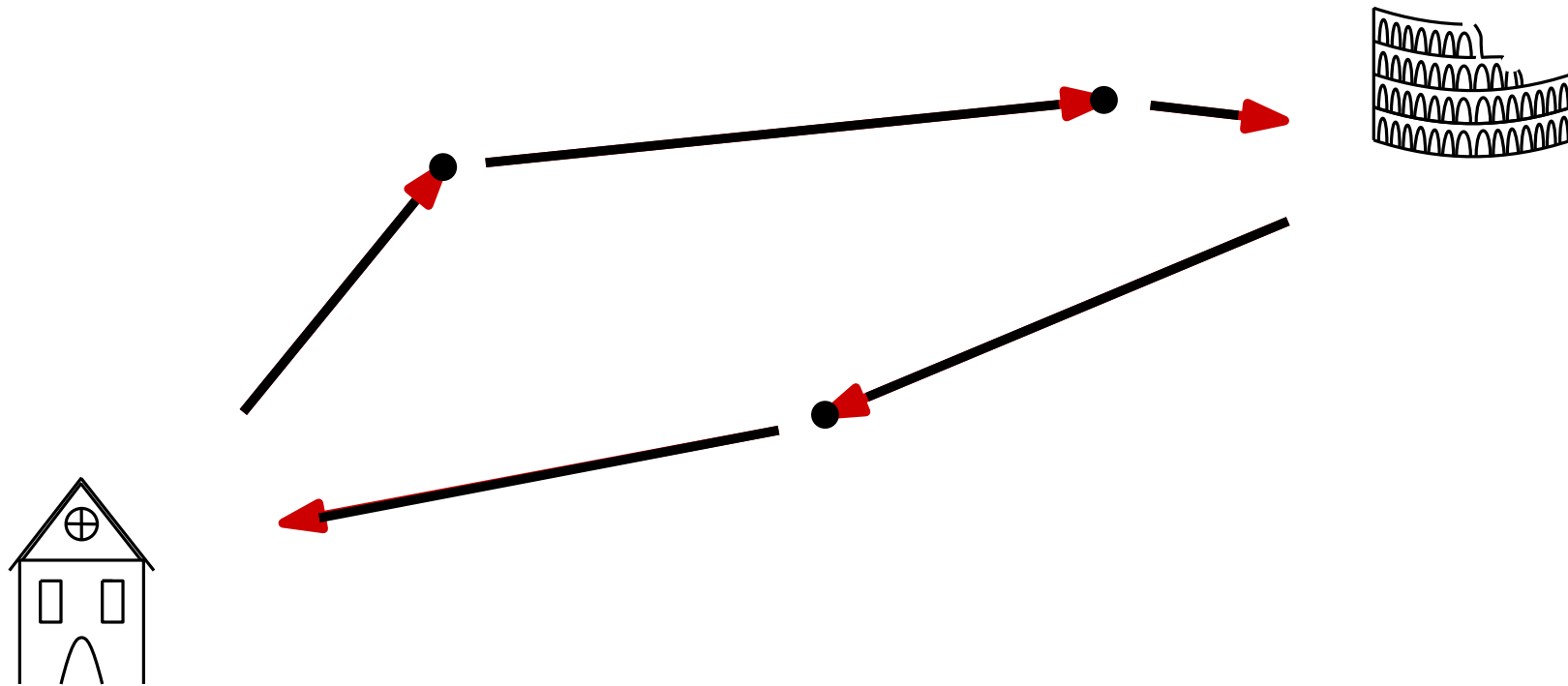
If the graph has a cycle:



If the cycle does not contain both, Home and Rome, than by the same argument as for the tree there is no solution

Subtask2: $N \leq M$

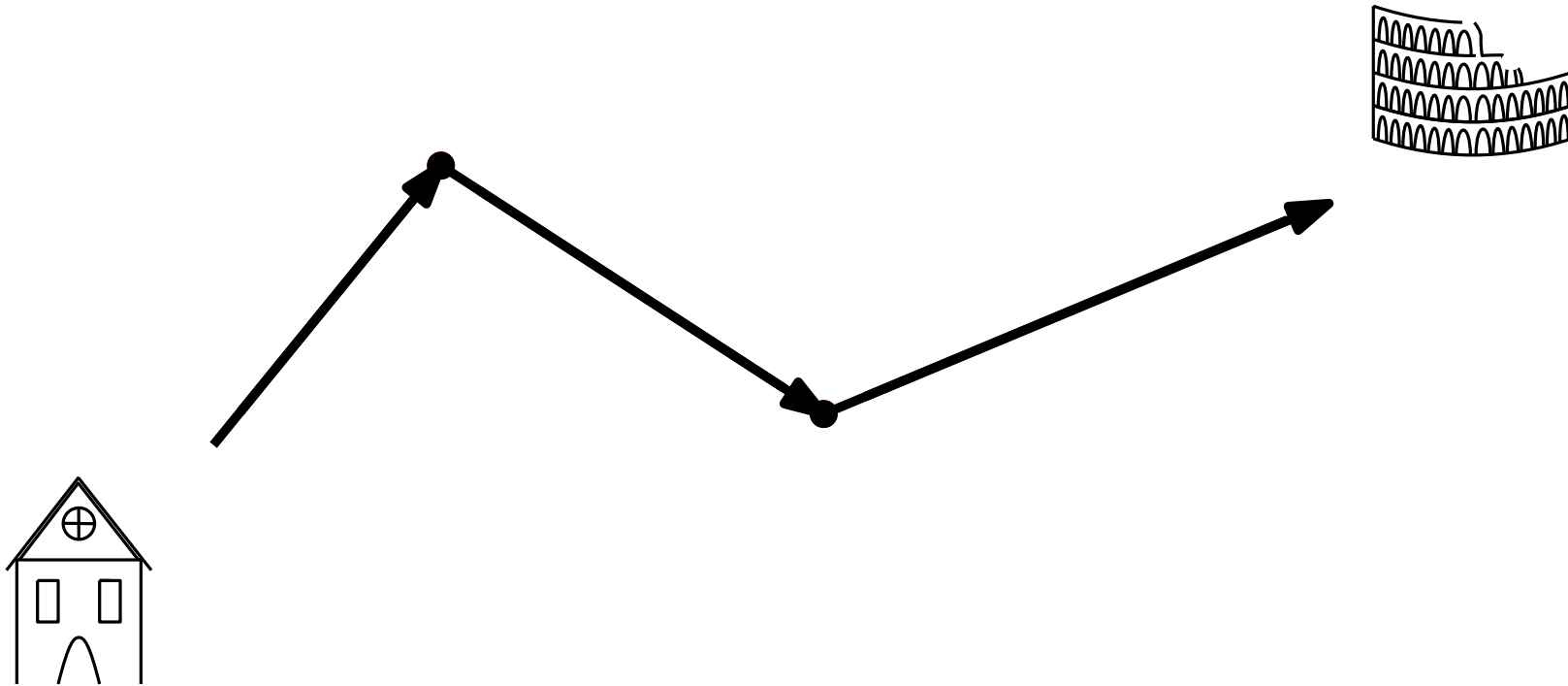
If the graph has a cycle:



If the cycle goes through both Home and Rome, then we can direct the cycle to form a directed cycle so that the shorter path goes from Home to Rome

Full solution

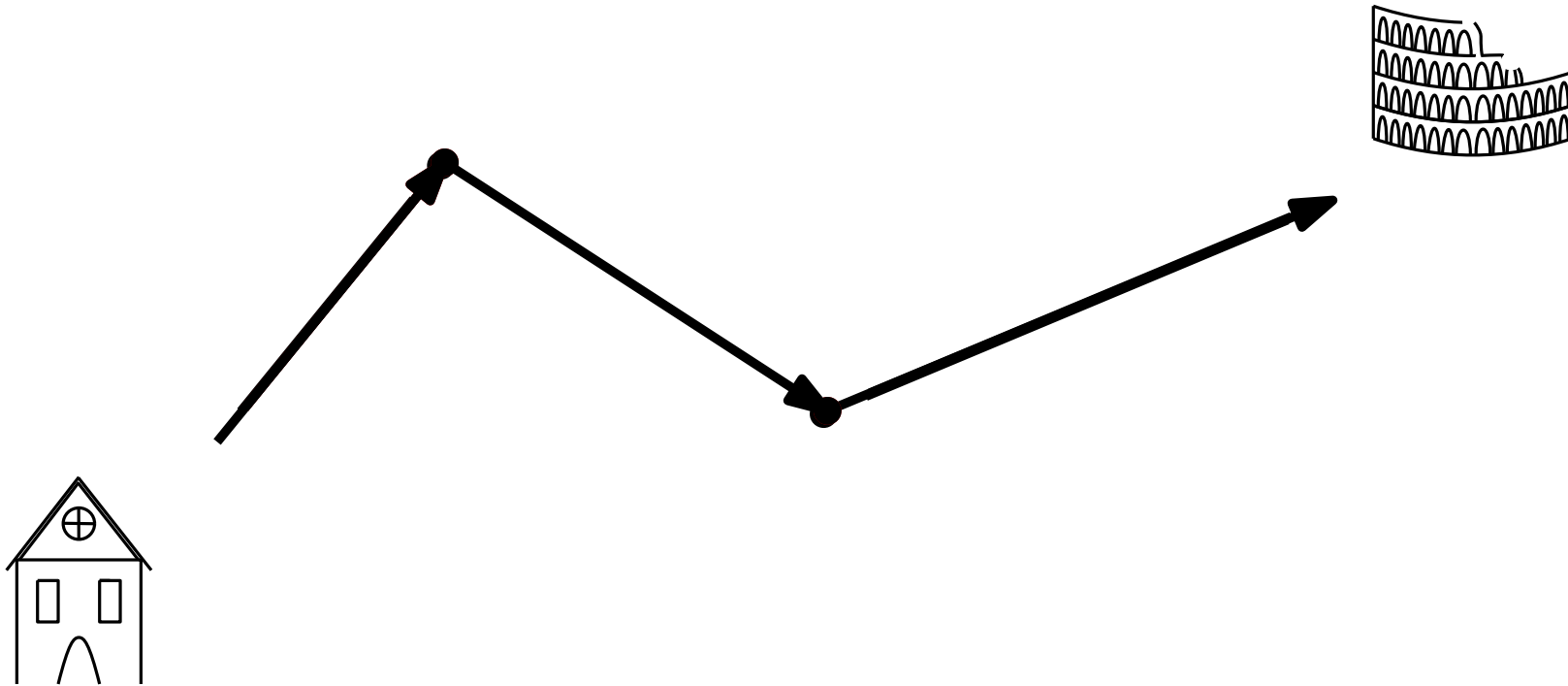
Step 1: Find a shortest path from Home to Rome



Direct these edges to go towards Rome

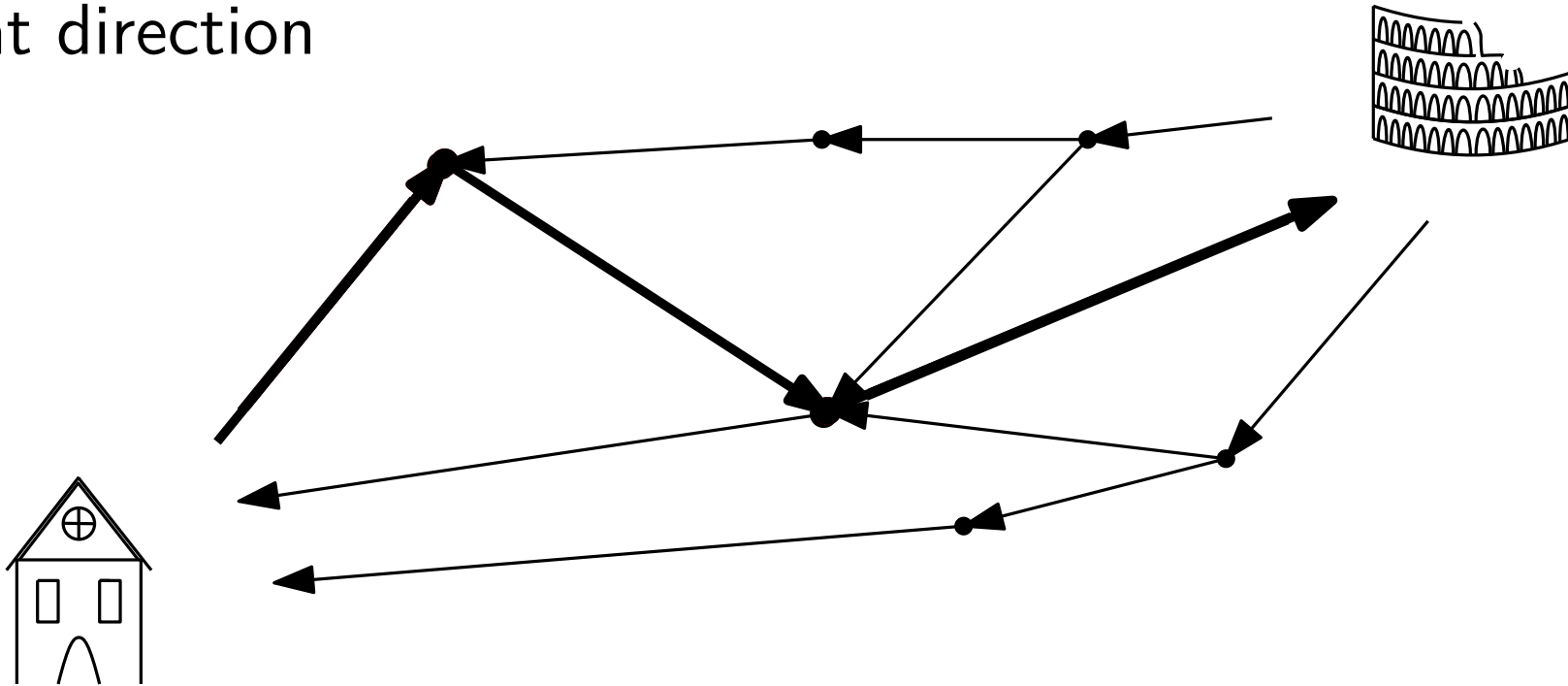
Full solution

Step 2: Direct the rest of the edges such that we can still reach Home



Full solution

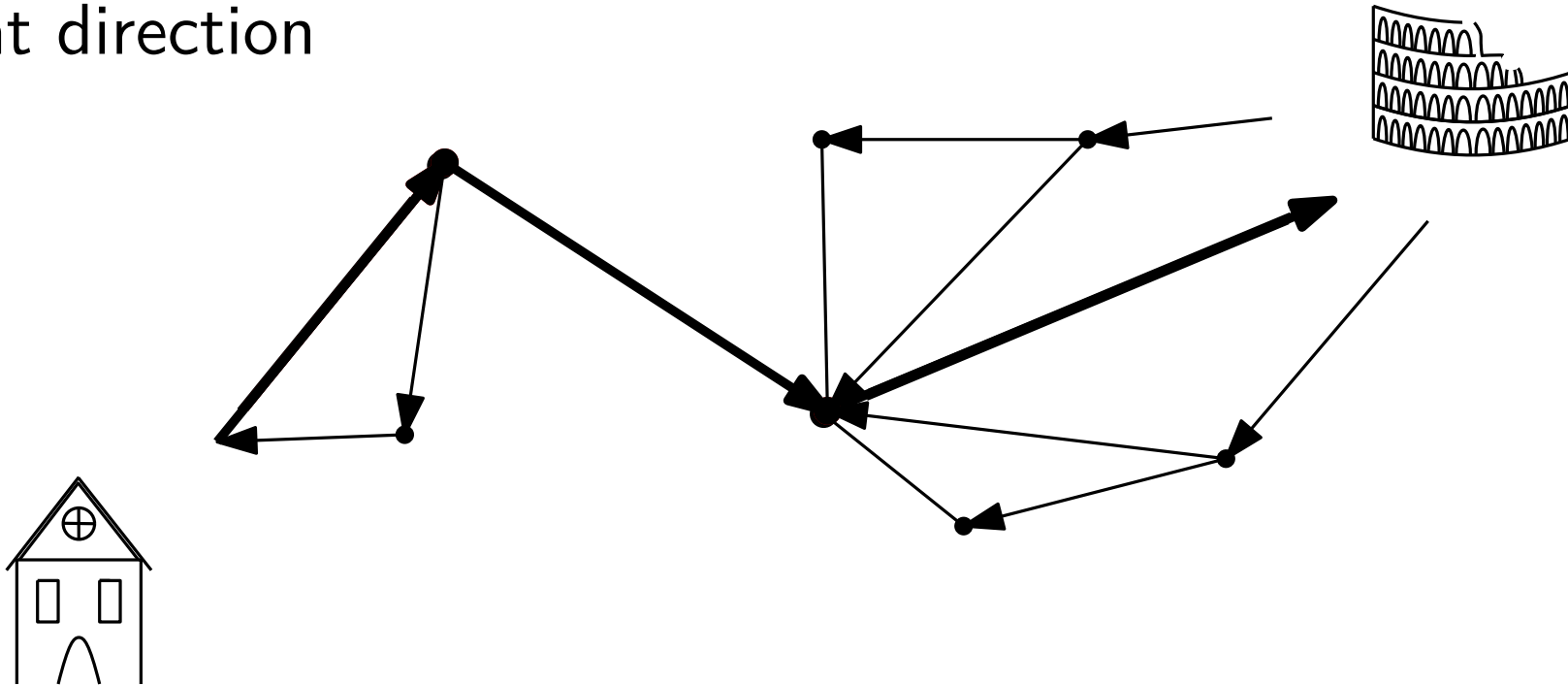
Step 2: Direct the rest of the edges such that we can still reach Home
Start a BFS in Rome, directing the edges as you encounter them, except for edges on the shortest path that are already directed and can only be used in that direction



If this reaches home, then output the edges you encountered in the direction you assigned to them and any other edges you can orient arbitrarily

Full solution

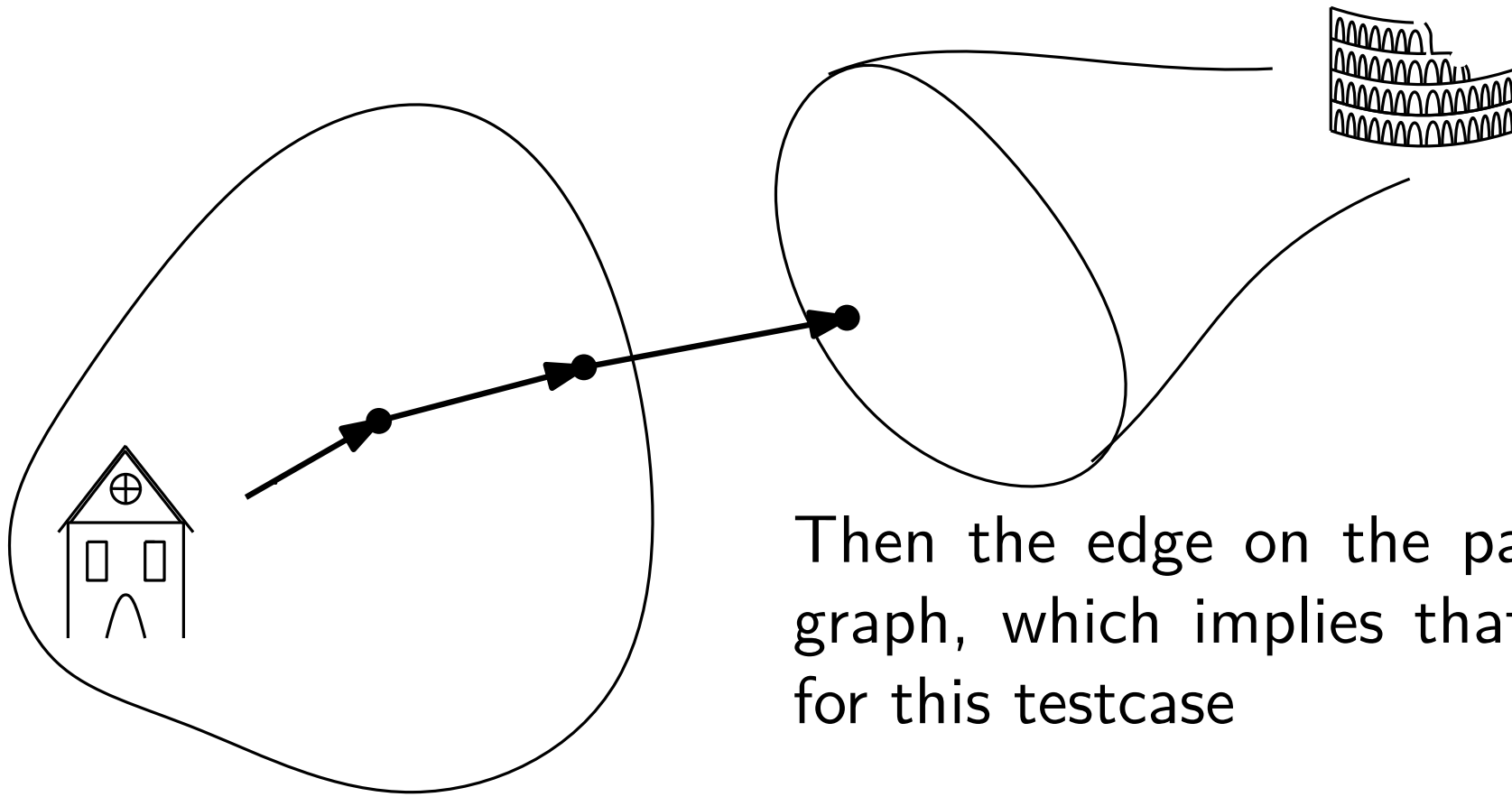
Step 2: Direct the rest of the edges such that we can still reach Home
Start a BFS in Rome, directing the edges as you encounter them, except for edges on the shortest path that are already directed and can only be used in that direction



Else there is a vertex on the shortest path such that no vertex closer to Home can be reached from Rome in this graph

Full solution

Step 2: Direct the rest of the edges such that we can still reach Home



Then the edge on the path is a bridge in the graph, which implies that there is no solution for this testcase